

Clock

Domyślnym źródłem czasu w ROS jest czas systemowy z komputera. Niestety w przypadku, gdy chcemy odtwarzać dane w czasie przyspieszonym, spowolnionym lub mieć większą kontrolę nad czasem konieczne jest zastosowanie symulowanego zegara czasu. W przypadku odtwarzania danych z czujnika ważniejsze jest by timestamp z jakim są zapisane dane odnosił się do czasu z jakim były one zbierane, a nie czasem systemu komputera na którym będą odtwarzane dane np. położenie enkodera w danych chwilach czasowych. W tej sytuacji dla różnych zegarów czasu wyznaczone prędkości obrotowe na podstawie zebranych danych byłyby różne, gdyby dla tych samych danych odnosił się do nich inny timestamp.

Do użycia symulowanego czasu przez node'a konieczne jest wcześniejsze ustawienie parametru w ROS: \

```
/use_sim_time na True
```

W ROS w danej chwili może być aktywny maksymalnie tylko jeden serwer czasu. Serwerem czasu może być dowolny node, który będzie publikował na topicu **/clock** wiadomość typu **roslib/Clock**.

Podczas odtwarzania rosbaga może okazać się, że brakuje źródła czasu. W takiej sytuacji polecenie należy uruchomić używając \

```
--clock
```

```
In [1]: import rospy
rospy.init_node("data_node")
```

Odczytywanie aktualnego czasu w ROS.

```
In [12]: # Zwracany obiekt rospy.rostime.Time
now = rospy.get_rostime()
print(now)
print(type(now))
```

```
1679403311943055868
<class 'rospy.rostime.Time'>
```

```
In [13]: # Zwracany czas w sekundach
now = rospy.get_time()
print(now)
print(type(now))
```

```
1679403312.4224575
<class 'float'>
```

```
In [14]: # Zwracany obiekt rospy.rostime.Time
now = rospy.Time.now()
print(now)
print(type(now))
```

```
1679403312943248033
<class 'rospy.rostime.Time'>
```

Dla symulowanego czasu wartość może wynosić 0 co oznacza, że czas nie jest znany.

```
In [17]: # Uśpienie na określony czas w sekundach
d = rospy.Duration(2)
rospy.sleep(d)
```

Bag

Jest to format danych przechowujący informacje o wiadomościach, które zostały wybrane do zarejestrowania w trakcie działania jakiegoś procesu np. monitorowanie zmiany położenia w czasie, zbieranie danych z czujników. Zebrane dane mogą być później użyte do testowania różnych algorytmów np. do wyznaczania odometrii na podstawie fuzji danych z sensorów, testowania różnych parametrów algorytmów do mapowania terenu.

Do utworzenia bag'a wykorzystywane jest narzędzie rosbag umożliwiające rejestrowanie wiadomości ROS. Takie podejście umożliwia odtwarzanie i dostarczanie danych do systemu w taki sam sposób w jaki się pojawiają przy użyciu node'ów.

Niektóre przykłady działania rosbaga wymagają uruchomienia symulacji w gazebo (*roslaunch multirobot_nav multi_turtlebot3.launch*) lub skryptu 5_1. Dostępne komendy dla polecenia rosbag.

```
In [14]: !rosbag --help
```

Usage: rosbag <subcommand> [options] [args]

A bag is a file format in ROS for storing ROS message data. The rosbag command can record, replay and manipulate bags.

Available subcommands:

check	Determine whether a bag is playable in the current system, or if it can be migrated.
compress	Compress one or more bag files.
decompress	Decompress one or more bag files.
decrypt	Decrypt one or more bag files.
encrypt	Encrypt one or more bag files.
filter	Filter the contents of the bag.
fix	Repair the messages in a bag file so that it can be played in the current system.
help	
info	Summarize the contents of one or more bag files.
play	Play back the contents of one or more bag files in a time-synchronized fashion.
record	Record a bag file with the contents of specified topics.
reindex	Reindexes one or more bag files.

Rosbag record - rejestrowanie danych

Szczegółowe informacje o dostępnych możliwościach polecenia można znaleźć na stronie.

<http://wiki.ros.org/rosbag/Commandline> (<http://wiki.ros.org/rosbag/Commandline>)

Rejestrowane mogą być nie tylko wszystkie dane jednocześnie tak ja się pojawiają w systemie, ale mamy możliwość wybrania sposobu jak mają być zarejestrowane. Można zarejestrować m.in.:

- pojedyncze wiadomości (oddzielne nazwy topiców)
- wiadomości spełniające określone kryteria (-e (regex), -x (exclude_regex) z podanym kryterium filtracji)
- rosbaga o określonym czasie trwania (--duration)
- rosbaga o określonym rozmiarze (--size)
- podział baga na mniejszy plik gdy czas trwania lub rozmiar zostanie przekroczony (--split)
- rosbaga o określonym prefixie w nazwie (-o) oraz o podanej nazwie (-O)
- maksymalna liczba zarejestrowanych danych, starsze pliki będą kasowane (--max-splits)
- topici z określonego node'a (--node)

```
In [7]: # Pomoc do polecenia od rejestrowania danych
!rosbag record --help
```

Usage: rosbag record TOPIC1 [TOPIC2 TOPIC3 ...]

Record a bag file with the contents of specified topics.

Options:

-h, --help	show this help message and exit
-a, --all	record all topics
-e, --regex	match topics using regular expressions
-p, --publish	publish a msg when the record begin
-x EXCLUDE_REGEX, --exclude=EXCLUDE_REGEX	exclude topics matching the follow regular expression (subtracts from -a or regex)
-q, --quiet	suppress console output
-o PREFIX, --output-prefix=PREFIX	prepend PREFIX to beginning of bag name (name will always end with date stamp)
-O NAME, --output-name=NAME	record to bag with name NAME.bag
--split	split the bag when maximum size or duration is reached
--max-splits=MAX_SPLITS	Keep a maximum of N bag files, when reaching the maximum erase the oldest one to keep a constant number of files.
--size=SIZE	record a bag of maximum size SIZE MB. (Default: infinite)
--duration=DURATION	record a bag of maximum duration DURATION in seconds, unless 'm', or 'h' is appended.
-b SIZE, --bufsize=SIZE	use an internal buffer of SIZE MB (Default: 256, 0 = infinite)
--chunksize=SIZE	Advanced. Record to chunks of SIZE KB (Default: 768)
-l NUM, --limit=NUM	only record NUM messages on each topic
--node=NODE	record all topics subscribed to by a specific node
-j, --bz2	use BZ2 compression
--lz4	use LZ4 compression
--tcpnodelay	Use the TCP_NODELAY transport hint when subscribing to topics.
--udp	Use the UDP transport hint when subscribing to topics.
--repeat-latched	Repeat latched msgs at the start of each new bag file.

W najprostszej wersji można zarejestrować wszystkie pojawiające się topic'i w systemie

rosvag record -a

```
In [ ]: !rosvag record -a
```

Tylko wybrane:

rosvag record *nazwa_topic nazwa_topic* ...

```
In [20]: !rosvag record /turtle1/cmd_vel
```

```
[ INFO] [1679403856.051557709]: Subscribing to /turtle1/cmd_vel
[ INFO] [1679403856.055271559]: Recording to '2023-03-21-13-04-16.bag'.
^C
```

Zapis rosvaga o określonym czasie trwania.

```
In [11]: !rosvag record --duration=10 /turtle1/cmd_vel
```

```
[ INFO] [1679403916.858284498]: Subscribing to /turtle1/cmd_vel
[ INFO] [1679403916.862223050]: Recording to '2023-03-21-13-05-16.bag'.
```

Z dzieleniem rosvaga, gdy przekroczy dany rozmiar lub długość czasu rejestracji.

```
In [11]: !rosvag record --split --size=512 /tb3_2/camera/rgb/image_raw
```

```
[ INFO] [1679404446.152996945]: Subscribing to /tb3_2/camera/rgb/image_raw
[ INFO] [1679404446.448817822, 302.745000000]: Recording to '2023-03-21-13-14-06_0.bag'.
[ INFO] [1679404482.491935446, 327.612000000]: Closing '2023-03-21-13-14-06_0.bag'.
[ INFO] [1679404482.501175943, 327.619000000]: Recording to '2023-03-21-13-14-42_1.bag'.
^C
```

Czas może być podany z typem jednostki. Brak jednostki odnosi się do sekund (m-minuty, h-godziny).

```
In [ ]: !rosvag record --split --duration=30m /turtle1/cmd_vel
```

Zapis tylko X ostatnich rosvagów. Każdy kolejny nadmiarowy powoduje usunięcie najstarszego zarejestrowanego rosvaga.

```
In [2]: !rosvag record --split --size 100 --max-splits 3 /tb3_2/camera/rgb/image_raw
```

```
[ INFO] [1679404539.968526369]: Subscribing to /tb3_2/camera/rgb/image_raw
[ INFO] [1679404540.194616344, 372.392000000]: Recording to '2023-03-21-13-15-40_0.bag'.
[ INFO] [1679404550.099191665, 378.146000000]: Closing '2023-03-21-13-15-40_0.bag'.
[ INFO] [1679404550.10009030, 378.146000000]: Recording to '2023-03-21-13-15-50_1.bag'.
[ INFO] [1679404559.848913220, 383.906000000]: Closing '2023-03-21-13-15-50_1.bag'.
[ INFO] [1679404559.849635039, 383.906000000]: Recording to '2023-03-21-13-15-59_2.bag'.
[ INFO] [1679404569.337502340, 389.709000000]: Closing '2023-03-21-13-15-59_2.bag'.
[ INFO] [1679404569.338257049, 389.710000000]: Recording to '2023-03-21-13-16-09_3.bag'.
[ INFO] [1679404578.45000488, 395.305000000]: Closing '2023-03-21-13-16-09_3.bag'.
[ INFO] [1679404578.501999964, 395.340000000]: Recording to '2023-03-21-13-16-18_4.bag'.
[ INFO] [1679404587.123161559, 401.006000000]: Closing '2023-03-21-13-16-18_4.bag'.
[ INFO] [1679404587.161020343, 401.036000000]: Recording to '2023-03-21-13-16-27_5.bag'.
^C
```

Zapis wszystkich topic'ów dla node'a o podanej nazwie /sterowanie

```
In [ ]: !rosvag record --node=circle_drive
```

Zarejestrowanie bag'a o podanej nazwie. Wykorzystany zostanie do dalszych ćwiczeń. Zarejestrowany zostanie z czasem trwania, gdyż przerwanie rejestrowania z poziomu jupyter notebook najczęściej wymaga reindeksowania.

```
In [10]: !rosvag record -O TSR.bag --duration=10 /robot1/cmd_vel /robot2/cmd_vel /robot3/cmd_vel
```

```
[ INFO] [1679405745.726392593]: Subscribing to /robot1/cmd_vel
[ INFO] [1679405745.737267516]: Subscribing to /robot2/cmd_vel
[ INFO] [1679405745.743361644]: Subscribing to /robot3/cmd_vel
[ INFO] [1679405745.990548722, 1399.964000000]: Recording to 'TSR.bag'.
```

Uwaga

W trakcie zapisu rosvaga przy niewłaściwym jego zamknięciu może zdarzyć się, że będzie miał on rozszerzenie active. Takiego rosvaga nie można odtworzyć, ale można dokonać reindeksowania, które rozwiązuje problem. Oryginalny rosvag zapisany zostanie jako kopia pod tą samą nazwą z rozszerzeniem .orig.bag.

```
In [20]: !rosvag reindex "2022-03-29-16-22-58.bag.active"
```

Skipping 2022-03-29-16-22-58.bag.active. Backup path 2022-03-29-16-22-58.bag.orig.active already exists.

Rosbag info - informacja o rosbagu

Zawiera podstawowe informacje o rosbagu tj. wersja, czas trwania, data i godzina rozpoczęcia i zakończenia, rozmiar, typy zarejestrowanych wiadomości i topic'i.

```
In [11]: !rosbag info "TSR.bag"
```

```
path:      TSR.bag
version:   2.0
duration:  10.0s
start:     Jan 01 1970 00:23:19.98 (1399.98)
end:       Jan 01 1970 00:23:29.96 (1409.96)
size:      5.8 MB
messages:  57350
compression: none [7/7 chunks]
types:     geometry_msgs/Twist [9f195f881246fdfa2798d1d3eebca84a]
topics:    /robot1/cmd_vel      19113 msgs    : geometry_msgs/Twist
           /robot2/cmd_vel      19077 msgs    : geometry_msgs/Twist
           /robot3/cmd_vel      19160 msgs    : geometry_msgs/Twist
```

Rosbag play - odtwarzanie rosbaga

Do odtwarzania rosbaga wykorzystanie zarejestrowany wcześniej TSR.bag.

```
In [26]: !rosbag play --help
```

Usage: rosbag play BAGFILE1 [BAGFILE2 BAGFILE3 ...]

Play back the contents of one or more bag files in a time-synchronized fashion.

Options:

```
-h, --help                show this help message and exit
-p PREFIX, --prefix=PREFIX
                           prefix all output topics
-q, --quiet                suppress console output
-i, --immediate            play back all messages without waiting
--pause                   start in paused mode
--queue=SIZE              use an outgoing queue of size SIZE (defaults to 100)
--clock                   publish the clock time
--hz=HZ                   use a frequency of HZ when publishing clock time
                           (default: 100)
-d SEC, --delay=SEC       sleep SEC seconds after every advertise call (to allow
                           subscribers to connect)
-r FACTOR, --rate=FACTOR  multiply the publish rate by FACTOR
-s SEC, --start=SEC       start SEC seconds into the bag files
-u SEC, --duration=SEC    play only SEC seconds from the bag files
--skip-empty=SEC          skip regions in the bag with no messages for more than
                           SEC seconds
-l, --loop                 loop playback
-k, --keep-alive           keep alive past end of bag (useful for publishing
                           latched topics)
--try-future-version      still try to open a bag file, even if the version
                           number is not known to the player
--topics                  topics to play back
--pause-topics             topics to pause on during playback
--bags=BAGS               bags files to play back from
--wait-for-subscribers    wait for at least one subscriber on each topic before
                           publishing
--rate-control-topic=RATE_CONTROL_TOPIC
                           watch the given topic, and if the last publish was
                           more than <rate-control-max-delay> ago, wait until the
                           topic publishes again to continue playback
--rate-control-max-delay=RATE_CONTROL_MAX_DELAY
                           maximum time difference from <rate-control-topic>
                           before pausing
```

Przed odtworzeniem rosbaga należy zrestartować symulację turtlesim do poziomu bazowego i dodać roboty:

```
In [12]: !rosservice call /reset
!rosservice call /kill turtle1
!rosservice call /spawn 5.5 4 0 robot1
!rosservice call /spawn 5.5 2.5 0 robot2
!rosservice call /spawn 5.5 1 0 robot3
```

```
name: "robot1"
name: "robot2"
name: "robot3"
```

Odtwarzanie całego rosbaga o podanej nazwie

```
In [14]: !roscat play "TSR.bag"
```

```
[ INFO] [1679405840.367244220]: Opening TSR.bag

Waiting 0.2 seconds after advertising topics... done.

Hit space to toggle paused, or 's' to step.
[RUNNING] Bag Time: 1409.961217 Duration: 9.976217 / 9.978000
Done.
```

Po zrestartowaniu i uruchomieniu symulacji z odtworzonymi danymi roboty poruszały się po okręgu. Jednak nie zawsze zarejestrowane sterowanie przekazane ponownie do tego samego robota pozwoli uzyskać taki sam efekt. Wpływ na różnice ma czas wysłania komend.

Zapis danych z rosbaga do pliku .yaml

Do zarejestrowania danych z poziomu terminala w lokalizacji z rosbagiem TSR.bag wywołać polecenie, które natychmiast odtworzy wszystkie zarejestrowane wiadomości w tym wymaganej /robot1/cmd_vel.

```
roscat play --immediate TSR.bag
```

```
In [17]: !rostopic echo /robot1/cmd_vel | tee velocities.yaml
```

Python Odczyt i analiza danych

Link do wiki zawierającego więcej szczegółowych informacji i przykładów. <http://wiki.ros.org/roscat/Cookbook> (<http://wiki.ros.org/roscat/Cookbook>)

Do obsługi rosbagów w Python służy biblioteka roscat.

```
In [18]: import roscat
```

```
In [19]: # utworzenie obiektu Bag, obsługuje dane zawarte w rosbagu.
bag_name = "TSR.bag"
bag = roscat.Bag(bag_name)
```

```
In [20]: # wyświetlenie surowych danych o topic'ach i ich typach
print(bag.get_type_and_topic_info())
```

```
TypesAndTopicsTuple(msg_types={'geometry_msgs/Twist': '9f195f881246fdfa2798d1d3eebca84a'}, topics={'/robot1/cmd_vel': TopicTuple(msg_type='geometry_msgs/Twist', message_count=19113, connections=1, frequency=None), '/robot2/cmd_vel': TopicTuple(msg_type='geometry_msgs/Twist', message_count=19077, connections=1, frequency=None), '/robot3/cmd_vel': TopicTuple(msg_type='geometry_msgs/Twist', message_count=19160, connections=1, frequency=None)})
```

```
In [21]: # Wyświetlenie zarejestrowanych topic'ów
for data in bag.get_type_and_topic_info()[1].keys():
    print(data)
```

```
/robot1/cmd_vel
/robot2/cmd_vel
/robot3/cmd_vel
```

```
In [22]: # Wyświetlenie listy topic'ów wraz z typami
for data in bag.get_type_and_topic_info()[1].keys():
    print(data + " " + bag.get_type_and_topic_info()[1][data][0])
```

```
/robot1/cmd_vel geometry_msgs/Twist
/robot2/cmd_vel geometry_msgs/Twist
/robot3/cmd_vel geometry_msgs/Twist
```

```
In [23]: # Odczyt pierwszej wiadomości z rosbaga. Metoda read_messages() odpowiada za odczyt kolejnych
# zarejestrowanych wiadomości z każdą kolejną wykonaną iteracją pętli for. Odczytane wartości
# po wykonaniu iteracji zapisywane są kolejno w zmiennych:
# - topic (nazwa topicu typu str)
# - msg (wiadomość ROS, dla cmd_vel jest to typ geometry_msgs/Twist)
# - t (czas w ROS, w którym zarejestrowana była wiadomość typu rostime.Time)

tsr_roscat = roscat.Bag("TSR.bag")
```

```

for topic, msg, t in tsr_rosbag.read_messages():
    if topic == "/robot1/cmd_vel":
        print(topic)
        print(type(topic))
        print("-----")
        print(msg)
        print(type(msg))
        print("-----")
        print(t)
        print(type(t))
        break

```

```

/robot1/cmd_vel
<class 'str'>
-----
linear:
  x: 1.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0
<class 'tmpgqc9_szz._geometry_msgs__Twist'>
-----
1399985000000
<class 'rospy.rostime.Time'>

```

Python - logowanie danych

Informacje wyświetlane i publikowane przez node'y w trakcie pracy systemu. Pozwalają zorientować się co aktualnie jest wykonywane i czy wszystko działa prawidłowo.

```

In [ ]: rospy.logdebug(msg, *args) # informacje do debugowania, niepotrzebne
        # dla użytkownika w trakcie prawidłowej pracy
        rospy.logwarn(msg, *args) # ostrzeżenie
        rospy.loginfo(msg, *args) # informacja
        rospy.logerr(msg, *args) # błąd możliwy do rozwiązania przy pomocy recovery
        rospy.logfatal(msg, *args) # błąd krytyczny, nie da się go naprawić

```

RQT_BAG

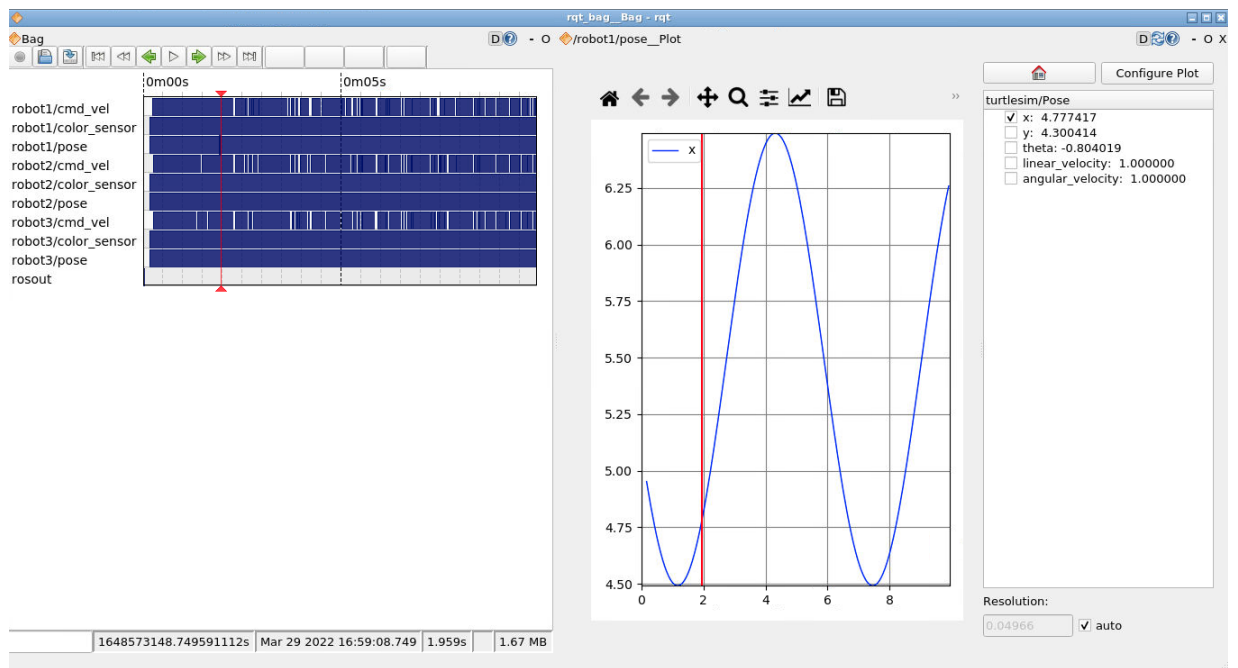
Narzędzie graficzne do nagrywania i wizualizacji danych z rosbaga.

Uruchomienie:

```

In [ ]: !roslaunch rqt_bag rqt_bag

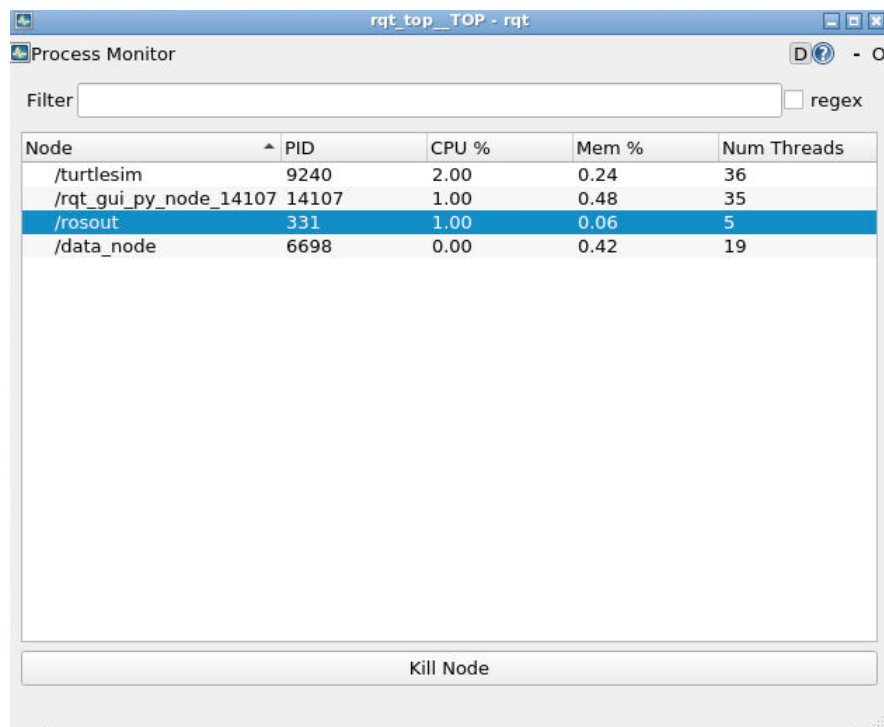
```



RQT_TOP

Do monitorowania aktualnie działających nodów. Możliwość filtracji i wyłączania niektórych node'ów.

```
In [ ]: !roslaunch rqt_top rqt_top
```

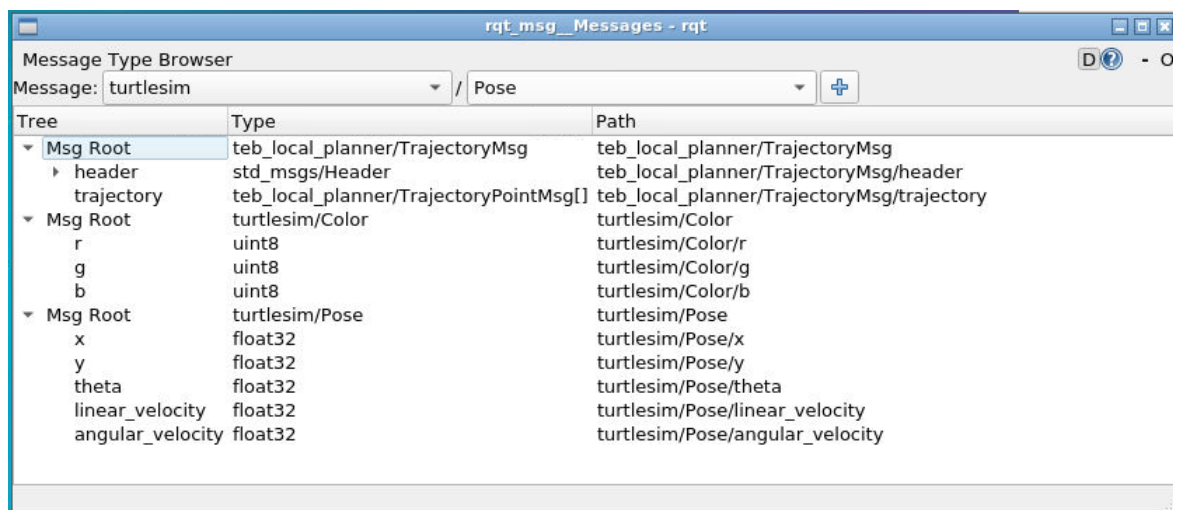


RQT_MSG

Graficzne narzędzie do podglądu zainstalowanych w systemie dostępnych z punktu widzenia ROS typów wiadomości dla topic'ów. Umożliwia podgląd struktury wiadomości.

Uruchomienie:

```
In [ ]: !roslaunch rqt_msg rqt_msg
```

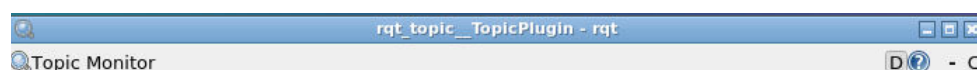


RQT_TOPIC

Graficzne narzędzie do monitorowania wybranych topic'ów, które są aktywne w systemie.

Uruchomienie:

```
In [2]: !roslaunch rqt_topic rqt_topic
```



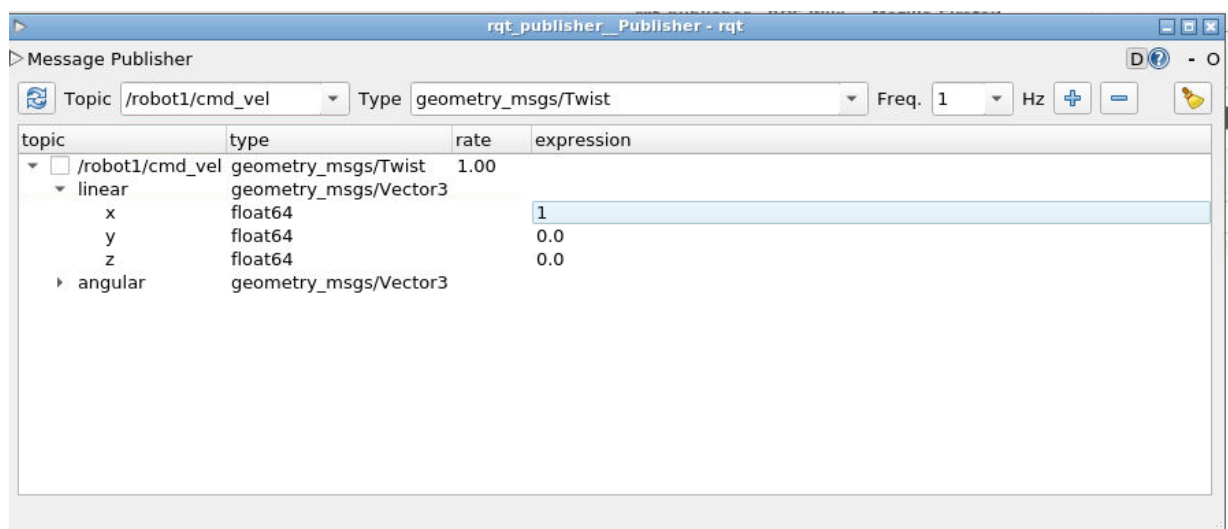
Topic	Type	Bandwidth	Hz	Value
▸ ☐ /robot1/color_sensor	turtlesim/Color			not monitored
▸ ☒ /robot1/pose	turtlesim/Pose	1.25KB/s	62.51	
▸ ☐ /robot2/color_sensor	turtlesim/Color			not monitored
▾ ☒ /robot2/pose	turtlesim/Pose	1.25KB/s	62.50	
angular_velocity	float32			0.0
linear_velocity	float32			0.0
theta	float32			0.1584441065788269
x	float32			5.894249439239502
y	float32			2.533839225769043
▸ ☐ /robot3/color_sensor	turtlesim/Color			not monitored
▸ ☐ /robot3/pose	turtlesim/Pose			not monitored
▸ ☐ /rosout	rosgraph_msgs/Log			not monitored
▸ ☐ /rosout_agg	rosgraph_msgs/Log			not monitored

RQT_PUBLISHER

Narzędzie pozwalające na publikowanie wiadomości z interfejsu graficznego. Do wyboru jest nazwa topicu, typ i częstotliwość z jaką ma być wysyłana wiadomość. Po kliknięciu prawym przyciskiem na wiadomość można wysłać ją tylko raz do robota.

Uruchomienie:

```
In [80]: !roslaunch rqt_publisher rqt_publisher
```

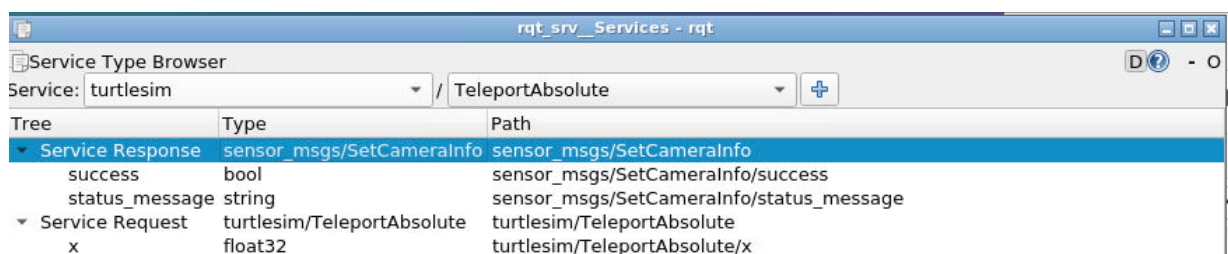


RQT_SRV

Graficzne narzędzie do podglądu dostępnych w systemie typów wiadomości serwisowych z podziałem na zapytania (request) i odpowiedzi (response).

Uruchomienie:

```
In [ ]: !roslaunch rqt_srv rqt_srv
```



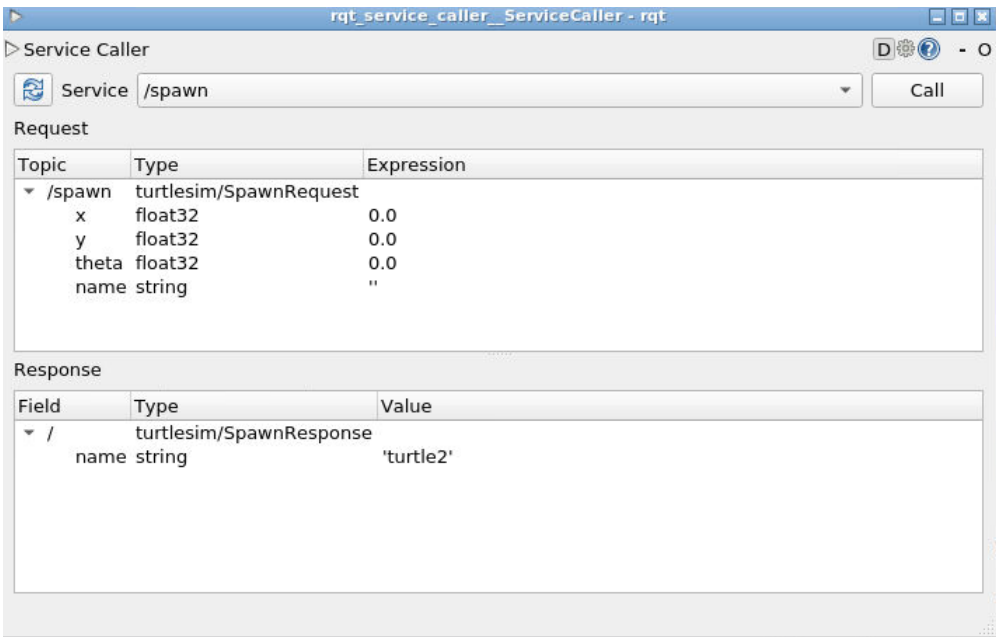
y	float32	turtlesim/TeleportAbsolute/y
theta	float32	turtlesim/TeleportAbsolute/theta
Service Response	turtlesim/TeleportAbsolute	turtlesim/TeleportAbsolute

RQT_SERVICE_CALLER

Narzędzie graficzne do wysyłania zapytań klienta na serwer.

Uruchomienie:

```
In [ ]: !roslun rqt_service_caller rqt_service_caller
```



RQT_LOGGER

Graficzne narzędzie do podglądu dostępnych w systemie dla różnych nodów loggerów danych na różnym poziomie błędów og krytycznych, po debugowanie czy zwykłe informacje.

Uruchomienie:

```
In [ ]: !roslun rqt_logger_level rqt_logger_level
```

